Connecting the app to the backend

HeatGuard AI · from the backend team to the frontend developer · 12 November 2026

In short

The backend now has real data for every worker and supervisor screen you built. Nothing needs redesigning.

The work is: **add a login screen**, **swap the sample data for real data, screen by screen**, **add three small things the backend needs** (clock-in buttons, a background "still here" signal, and optionally the wristband), and **make a few wording changes**: three agreed for privacy, and some because the occupational health part is not being built.

The work at a glance

Area	What to do	Size
Set-up	One place in the code that talks to the backend, with the login token and error handling	Small
Login	New screen: worker or staff ID plus a PIN; then open the right part of the app for that person	Small
Worker screens	Replace sample data on the 7 worker screens with real data	Medium
New in the worker app	Start shift / break buttons; a quiet "still here" signal every 5 minutes; wristband (optional)	Small to medium
Supervisor pages	Today, Shift plan and Measures each load from one backend call	Medium
Wording changes	Three small text changes agreed for privacy	Small
No occupational health	Remove promises of a nurse, hide "Request a review", drop the escalation outcome	Small

1. Set-up

- Add a setting for the backend address (for example `VITE_API_URL`) and one module that every screen uses to call it.
- Every call carries the login token. Handle errors in one place: "not logged in" sends the person to login; "not allowed" and other errors come with a readable message you can show.
- Swap the sample data one screen at a time. Small converter functions that turn backend answers into your existing types mean your components barely change.
- The backend sends **codes and times, not sentences**: for example `ha` instead of "Hausa", `high_priority` instead of "high", a crew ID instead of "A", and times in UTC. Your app turns these into words, in the site's timezone. This also makes translation easier.
- Use the device clock instead of the demo clock, and keep the dev panel for development only.

Your app today	The backend uses
English, Hausa, Yoruba, Igbo, Pidgin	`en`, `ha`, `yo`, `ig`, `pcm`
Precaution "high"	`high_priority` (also `standard`, `enhanced`)

Your app today	The backend uses
Crew A to D	Crew IDs and names from the backend
Age "40 to 54"	`40_54` (also `prefer_not_to_say`, `under_25`, `25_39`, `55_plus`)

> **The first request can be slow**
>
> The backend and the AI engine run on Render's free plan, which puts them to sleep after about 15 minutes without use. The first request after that can take 30 to 60 seconds (up to about two minutes if both are asleep); after that, requests are fast.
>
> Please let requests wait up to about 90 seconds before treating them as failed, and if one takes more than about 3 seconds, show a friendly **"Waking up the server..."** message instead of an error. On demo day the backend team keeps both awake.

2. Login

- A new screen with two boxes: **ID** (for example W-0412 or S-001) and **PIN** (4 to 6 digits).
- Send them to `POST /auth/login`. You get a token for calls (valid 15 minutes) and a second token to get a new one (valid 7 days, `POST /auth/refresh`).
- Wrong PIN: always show "Invalid ID or PIN". After 5 wrong tries the account pauses for 5 minutes, with the same message on purpose.
- After login, ask `GET /me` who this is and open the right part of the app: workers to `/worker`, supervisors to `/supervisor`. The role switcher can go (or stay for development).
- Demo logins: supervisor `S-001`, workers `W-0301` to `W-0508`, all with PIN `1234`.

3. Worker screens

Screen	What changes	Backend call
Welcome and consent	Language and the three sharing switches are saved on the server. First-time check: if the worker has never chosen, show this screen	`PATCH /me/preferences`, `PUT /me/consents`, `GET /workers/:id/consents`
Health profile	Load and save the answers (as codes). Show precautions from the server. Without the health switch on, saving is refused: show a friendly message. Hide "Request a review"	`GET` / `PUT /workers/:id/health-profile`, `GET /me`
Home	Risk level, action, reasons, next break, 6-hour forecast, other hazards (dust, storms) and the water count all come from the server	`GET /me/risk`, `GET` / `POST /me/water`
What to do	Keep your own advice for each level; show the engine's action as the headline. "Who" compares the worker's level with their crew's. "I understand" is saved on the server	`GET /me/risk`, `POST /recommendations/:id/interventions`
Report symptoms	Real emergency number. "Alert first aider" and "Send" go to the server; the alert always gets through, details only if the worker allowed sharing	`POST /me/first-aid-alert`, `POST /me/symptom-reports`
My data	Switches, "Who looked at your data", Download and Delete all use the server. Hide "Request a review"	`GET /me/access-log`, `GET /workers/:id/data-export`, `DELETE /workers/:id/health-profile`
Field mode	Save the latest risk and forecast while online, and use them when offline. Queue button presses made offline	(uses `GET /me/risk`)

> **Good to know on the Home screen**
>
> If the data is old you get `dataQuality: "degraded"`: show a small "no fresh heat data" note. If the risk service is down you get `source: "fallback"` and the level is at least High on purpose. Fields that the engine doesn't send yet ("possible" level, confidence) come back empty: just hide them.

4. Three new things the app needs to do

Start shift and break buttons

The risk engine needs to know how long each worker has been working and when they last rested. Please add four buttons, for example on Home: **Start shift**, **Start break**, **End break** and **End shift**. `GET /me` tells you the worker's shift and its ID; each button is one call (`POST /assignments/:id/clock-in`, `/break/start`, `/break/end`, `/clock-out`). Without these, the backend assumes the worker started on time and never rested: safe, but less accurate.

A quiet "still here" signal

While the app is open, send `POST /me/heartbeat` every 5 minutes with the phone's battery level (your `useBattery` hook already reads it). This has nothing to do with the wristband. It lets the supervisor see whose phone has gone quiet (counted as High) and who is low on battery.

Wristband (optional)

Only if `GET /me/wearable` says `enabled: true`. On Android Chrome, the app can connect to a standard Bluetooth heart-rate band, register it, and send about one reading a minute in batches (`PUT /me/wearable`, `POST /me/wearable/readings`). For the demo there is a simulator on the backend, so no real band is needed.

How to simulate the wristband

The backend has a band simulator. It signs in as a worker, pairs a pretend band and sends heart-rate and skin-temperature readings, exactly as your app would. Use it to build and test the band screens without any hardware.

1. Ask the backend team to turn on `WEARABLES_ENABLED` (or set it in the backend's `.env` and restart).
2. Turn on the worker's "Wearable band" switch in My data (this is the consent; without it the band is refused).
3. On the backend machine, run `npm run band:simulate -- W-0412 1234 --profile strained --minutes 30`. Profiles: `resting` (about 75 bpm), `working` (about 110 bpm), `strained` (climbing to about 155 bpm and 36 °C). Add `--follow` to keep sending a reading every minute.
4. What you should see: in the worker app, "Your band" shows as connected and `GET /me/wearable/readings` returns the chart data. On the supervisor's System health, "1 band connected", and never any readings.
5. To stop: turn the "Wearable band" switch off (this deletes the readings), or unpair.

5. Supervisor pages

Page	What changes	Backend call
Today	The whole page loads from one call: site heat now and hour by hour, workers by level (including "no recent data"), hazards, system health, needs attention, actions, today so far, crews and the last 7 days. Crew, activity and shift filters all work	`GET /sites/:id/today`
Needs attention buttons	"I am on my way" and "Mark resolved" for help requests; "Acknowledge" and "Reassign" for the other cards	`POST /help-requests/:id/acknowledge` or `/resolve`, `POST /recommendations/:id/interventions`
Shift plan	Load the current and suggested plans with the heat row and the red "too hot" cells from the backend. Edit and save; "Send to crews" applies a plan. The backend now owns the too-hot rule	`GET` / `PUT /sites/:id/plan`, `POST /sites/:id/plan/send`
Measures	All cards and charts for 7 days, 30 days or the season. Show a "provisional" badge where the backend says so, and "not tracked yet" for incidents	`GET /sites/:id/measures`

Supervisor screens receive codes for actions (for example `supervisor.worker_reduced_exposure`). The backend also sends English text, but mapping codes to your own wording makes translation possible.

6. Three wording changes (agreed for privacy)

Where	Change	Why
Supervisor Today, worker care card	Show only what the worker needs ("needs reduced exposure today"). Remove the level name ("High-priority precautions") and the word "precautions"	Supervisors must never learn a worker's health status (decision D1)
Health profile, "Getting used to the heat"	Move this question out of the health profile, or change the promise that the supervisor never sees the answers	The answer now feeds the work record the risk engine uses (decision D4)
Report symptoms, after sending	If sharing is off, say the supervisor was alerted but no details were kept	Decision D5: the alert always gets through, details only with consent

7. No occupational health screen: what changes

The occupational health part of the prototype is not being built. Everything else keeps working. These are the changes on the worker and supervisor screens.

Precaution levels

Only occupational health can set a worker's precaution level, and there is no screen for it. For the demo, the levels are set in the demo data: **W-0412 is high-priority** (supervisor sees "needs reduced exposure today") and **W-0384 is enhanced** ("needs earlier breaks today"). The backend team can change a level with a script. Nothing to build: just show what the backend returns.

Wording that promises a nurse

Screen	Today	Change to
Consent	"Lets occupational health set extra care for you"	"Lets HeatGuard give you extra care based on your health answers"
Consent	"Symptom reports to occupational health: So a nurse can follow up"	"Share symptom details: kept private; your supervisor only sees that you need help"
Health profile	"A nurse will ask you about it in private", "A nurse can check with you"	Remove the nurse promises; say answers are private and never shown to supervisors
Health profile	Button "Send to occupational health"	"Save my answers"
Health profile and My data	"Request a review" button	Hide it
My data	"Set by occupational health · 12 Oct"	"Set on 12 Oct"
Report	"Details go to occupational health only"	"Details stay private; your supervisor only sees that you need help"
Supervisor Today, help card	"...case escalated to the occupational health nurse" and the escalation outcome	"First aider notified"; remove the outcome line. "I am on my way" and "Mark resolved" still work

Do not build the `/occupational-health` route.

8. Things that will switch on by themselves later

- "Possible" levels and forecast confidence, when the AI engine adds them. Build the screens to show them when present.
- Advice in Pidgin, if the engine adds it. Until then Pidgin users get the advice in English.
- Wristband data affecting the risk level, when the engine supports it.
- Push notifications and SMS alerts are a later phase.

9. Before the demo

- The backend only accepts the app from `heat-guard-nine.vercel.app` and `localhost:5173`. If you use another address (for example a preview link), tell the backend team so it can be added.
- The backend team runs the demo data each demo day, so the workers are on shift, keeps the backend and engine awake, and turns on automatic updates every 15 minutes.
- Logins: supervisor `S-001`; workers `W-0412` (high-priority precautions) and `W-0384` (enhanced); PIN `1234`.

The full technical version, with every field name, is `FRONTEND_TASKS.md` in the same folder. The API can also be tried live at `/docs` on the backend.